

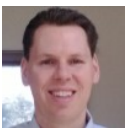


Professional Grade AI-Assisted Coding: Context Is Everything with BMAD and Spec Kit

By Bill Catlan

Advertisement:

Professional Grade AI-Assisted Coding: Context Is Everything with BMAD and Spec Kit



By **Bill Catlan**

Published in: **CODE Magazine: 2026 - July/August**

Last updated: June 24, 2026



AI coding assistant, such as Anthropic's Claude Code, and taking the generated code and inserting it into a codebase. This technique is generally done in a conversational style and is often referred to as “vibe coding.”

When vibe coding, the context being managed by the AI model during a single session (i.e., what the model “knows” about your ephemeral set of prompts and decisions at a given time) can become very large in scope and fairly unfocused as the developer continues a session while tackling a series of different tasks. Moreover, because the context reflects an interactive conversation—often with little or no structure—preserving the context is impossible. To help limit this kind of “context sprawl,” one easy technique is to clear the context session and begin a fresh session for a new task or feature.

However, when you clear a context session in this way, foundational decisions and other provenance (i.e., instructions and decisions formulated during the context session) are also lost. Moreover, any code that was generated during the cleared session now exists solely on its own, and does not capture any of the specifications, framework choices, or other factors and constraints that resulted in the generated code. This means that someone who wanted to subsequently revise some of the specifications or perhaps change the programming language framework or other aspects would not have the prior work to reference and build upon. Lastly, it also means that the AI is unconstrained if it were to be asked to regenerate similar code or modify existing code. When unconstrained, the AI model can draw from numerous sources to assess its options on how to proceed in



Share

Syntax Highlight Theme:

Kava Docs Dark



Published in:



Filed under:

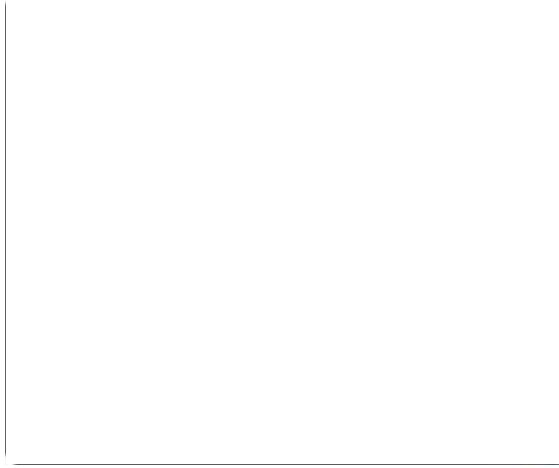
AI

Advertisement:



then cleared, are lost. This is the key deficiency of vibe coding which spec-driven development and context engineering seek to address.

Advertisement



Context engineering moves a developer from a vibe coding technique to a structured approach to generating code that preserves provenance using key artifacts. This allows context to be restored by other developers, reviewed by other stakeholders, or retrieved at any point if someone wanted to audit the inputs that generated a given codebase or simply wanted to continue building upon them. Different methodologies have emerged that offer different perspectives on how to effectively manage context and preserve provenance. **BMAD** (Breakthrough Method for Agile AI-Driven Development) and GitHub's **Spec Kit** are methodologies that offer different approaches to engineering context and preserving provenance. Being methodologies, they can be used with multiple AI assistants and models and are not tied to just one. After looking at some of the primitives and mechanics used for context engineering in general, both methodologies, along with their artifacts and philosophies, will be discussed to illustrate how they each allow the developer to



that offer different approaches to engineering context and preserving provenance. Being methodologies, they can be used with multiple AI assistants and models and are not tied to just one.

Context Primitives

Context primitive is almost a misnomer in the age of AI-assisted code generation for how applications get specified and built. Even the lowest level artifacts used for code generation are often quite sophisticated. Nonetheless, they are primitives in the sense that they are the low-level building blocks for your context session, and ultimately, your generated code. Whether the primitive is instructions in a markdown file, an agent, an MCP resource, or something else, it remains the case that the primitives exist to be assembled into a context session and better inform the generated outputs of that session. Moreover, as sophisticated as they often are, the primitives are typically of little use or meaning until they are assembled, and what is selected into a context session and what is left out is very important for obtaining reliable results.

Scoping Context

When you start to think about scoping context, the early steps are very intuitive. Perhaps you had the experience of your AI tool automatically identifying one or more open documents and automatically including them as part of your prompt's context? Perhaps that is what you wanted at the time and perhaps it wasn't.



engineering, it nonetheless begins to add a repeatable structure and intention to AI-assisted coding. The following techniques are the early primitives of methodologies like BMAD and Spec Kit. They include the following:

- **Selection.** Intelligently identify only the most relevant files for your session.
- **Compression & Compaction.** Summarize long conversation histories or technical logs to retain key decisions.
- **Ordering.** Place critical rules at the “top of the model's memory” and artifacts pertaining to the immediate task at the end in order to exploit the model's positional bias.
- **Isolation.** Split complex tasks across specialized “sub-agents” (e.g., one for planning, one for testing) for cleaner context.

These techniques are both a good way to get started with shaping the context of your vibe coding sessions and, as we'll see, also remain present if you decide to adopt a more sophisticated methodology, such as BMAD or Spec Kit.

Selection Is Primary

While each of the previous techniques are important, the first one to embrace and master is selection. Selection allows you to target important sections of your codebase and provenance files so that the artifacts that get loaded into the current context session are the ones that are most relevant to the current task. The artifacts that are allowed in the context session may be either foundational or reflect any intentional choices that you made for the



application, as compared to those that would clearly be limited to the UI front-end components or the API back-end components.

Selection allows you to target important sections of your codebase and provenance files so that the artifacts that get loaded into the current context session are the ones that are most relevant to the current task.

A Note About Selection and Isolation

It is often the case that selection techniques are used in concert with isolation techniques to limit certain tasks to specific directories, files, and other resources. Being primary, selection is employed to scope just about every process, including when isolating certain activities from other activities. Whether used in concert with each other or used separately, each represents a distinct capability for shaping the context session to allow the AI model to deliver results most in-line with your goals and expectations.

A tangible example of this can be shown using the `CLAUDE.md` file utilized by the Claude Code coding assistant. As a general matter, `CLAUDE.md` is a low-level primitive artifact that Claude Code will identify and, when placed in the root directory of the project, will load into every context session. `CLAUDE.md` is Claude Code's primary "memory" file.

Typically, you will find a `CLAUDE.md` file at the root of the project, perhaps like this:



However, it will often make sense for different decisions—such as programming language or framework information—to be made specifically for that component. As such, a project whose developers are interested in preserving this kind of intention, and not rely upon the model choosing something for itself, can indicate choices and opinions in a `CLAUDE.md` file place in a lower-level directory, such as like this:



```
/app/ui/CLAUDE.md  
/app/api/CLAUDE.md
```

These are then called “layered” rules, and Claude Code is smart enough to give them precedence. In addition, when setting up a context session, you can explicitly state the area of the code that you want to focus the session on, using direct instructions to the model, such as:



```
$ claude --ignore "!(/app/api/**) "
```

This is a hard boundary that tells `claude` to ignore everything that is not within the `/app/api` directory. Here we have “slipped” back into a vibe coding-like technique, which is often good for trying things out. But remember, this directive will not be preserved for review or use during future context sessions. For that, you will want to make an entry in the `CLAUDE.md` file within that directory. A simple example might look like this:



```
## Context Scope
```




A Note About Enforcing Locality

You might be tempted to just `cd` into the folder and run Claude. While Claude Code will see those files, it will still try to find the Git root to understand the full context. Using the `--ignore` or `@./` methods allows Claude to stay aware of the Git context while limiting the session context strictly to your specific sub-folder.

The Other Major Techniques

While selection may be the primary technique for scoping a context, the other techniques mentioned above are also very powerful.

As we will see at length in the sections on both the BMAD and the Spec Kit methodologies, the entire mechanisms for preserving provenance are built upon Compression & Compaction. We will see the specific artifact hierarchy that each methodology employs to capture the results of prompts and other inputs. While these notions are perhaps more intuitively understood as a kind of summarization, the general technical term is Compression & Compaction, primarily because those terms better represent the impact on the session context—namely, providing a succinct and efficient representation of the outputs of prior context sessions for use going forward.

Ordering is a straightforward concept and technique, and useful to keep in mind. While it may seem that earlier inputs would “fade” into the background as new context is developed, the fact of the matter is that



session than the particular requirements of the item at hand.

Lastly, *isolation* is all about addressing different questions using different agents with specialized knowledge or using a specialized agent or leveraging certain external resources that require specific skills by the agent, such as RAG or MCP resources. As such, isolation is perhaps best thought of as a targeted activity type, where selection is all about identifying and targeting specific input files. As the sidebar “*A Note About Selection and Isolation*” describes, the two techniques are very often used in concert as complementary techniques.

A Note About Selection and Isolation

It is often the case that selection techniques are used in concert with isolation techniques to limit certain tasks to specific directories, files, and other resources. Being primary, selection is employed to scope just about every process, including when isolating certain activities from other activities. Whether used in concert with each other or used separately, each represents a distinct capability for shaping the context session to allow the AI model to deliver results most in-line with your goals and expectations.

As we will see in the context engineering section below, BMAD harnesses a multi-agent approach, where specialized AI agents are asked to provide solutions to problems in their particular knowledge domain. For example, there is an Architect agent with which it is most appropriate to submit architecture level prompts and develop architecture artifacts. Similarly, there is a specialized Implementation agent,



Context Engineering

As described above, context engineering is the practice of honing or sharpening the baseline information that an AI model will use when suggesting solutions and generating code in response to a prompt. While the ability of models to handle more tokens in their context windows continues to expand every day, it is still important that the context session be focused and well-curated in order to achieve a strong command of what the AI model will do for you, and hence, professional-grade results.

Preserving Provenance

A major component of context engineering is preserving provenance. Again, preserving provenance is the structured approach to capturing the outputs of earlier, higher order, context sessions. In general, provenance means knowing the origin and history of something. With AI-assisted coding, documenting product features and expected outcomes, capturing design decisions, capturing foundational decisions (e.g., coding style guides or frameworks), enforcing architectural patterns, and specifying implementation decisions are all examples of areas where you want to preserve provenance (i.e., preserve outputs for later use by either your own project team or even other projects.) When you preserve your outputs in a well-organized set of artifacts, they can be picked up at any time to be reviewed, reused, tested, or refined by you, someone on your team, or anyone that you would like to invite to contribute to your code.

someone on your team, or anyone that you would like to invite to contribute to your code.

Incorporating this strategy into your AI-assisted coding practice is essentially what allows you to move from simple vibe coding (i.e., what in the past might have been referred to as free-form hacking) to a more rigorous process with discipline and structure. As with moving from improvised hacking to software design, moving from vibe coding to developing and preserving provenance using a structured methodology and well-defined set of artifacts has many long-term benefits. Those benefits include:

- **Predictability.** Foundational artifacts capture the core intentions and resulting design decisions you made, rather than leaving the AI model with wide latitude to infer choices, thereby producing expected outputs downstream; the result is cleaner context with many more intentions expressed, leaving less for the AI model to resolve on its own.
- **Reusability.** Other developers can read the higher order artifacts to better understand the framework and structure of your code; they can leverage these artifacts when they want to contribute to your project, or, they can download them and use them like reusable code libraries, providing a full set of design choices and other decisions that they can use in their own new projects.
- **Maintainability.** Because earlier outputs are preserved and always available for injection into a new context session, maintaining a codebase and adding new features is much more efficient and continuity is preserved when the new outputs



makes sense to do so.

- **Auditability.** Anyone can inspect the higher order artifacts, such as original product requirements and architecture decisions, and discover how and why the AI model is making certain choices during lower order activities like implementation; with vibe coding, those outputs would have been lost with a context session long ago, if they were ever truly developed in the first place.

Getting Started with BMAD and Spec Kit

The best way to get started with AI-assisted coding is, well, to get started! Both of the methodologies described here basically involve the same steps—install a code generation AI assistant CLI, select a model, and install the methodology's specific artifacts.

To get started with [BMAD](#), if you are starting from scratch, head over to your favorite web-based AI agent and enter the following prompt to get started:

PROMPT

```
How do I get started using the BMAD methodology  
with Claude Code?
```

For [Spec Kit](#), enter:

PROMPT

```
How do I get started using the Spec Kit  
methodology with Claude Code?
```

In both cases, you should be provided with the necessary steps for your platform.



versions of various coding assistants are more broadly available and are often ahead of the IDE plugins that may also be available in terms of features and capabilities. In my own case, I also opt to install the CLI on a remote, private AWS instance running Linux, from which I remotely mount the filesystem to use with my local IDE. See my article in the March/April 2026 issue of CODE Magazine, [Remote Debugging Using Mounted Code](#) to learn more about that powerful technique. This article uses the Claude Code CLI to drive both methodologies. When set up, a typical coding session looks like what is shown in **Figure 1**.

Advertisement

Note: the repositories containing all the code in this article can be found at <https://github.com/billcat-codemag/bmad-pong.git> and <https://github.com/billcat-codemag/speckit-pong.git>.

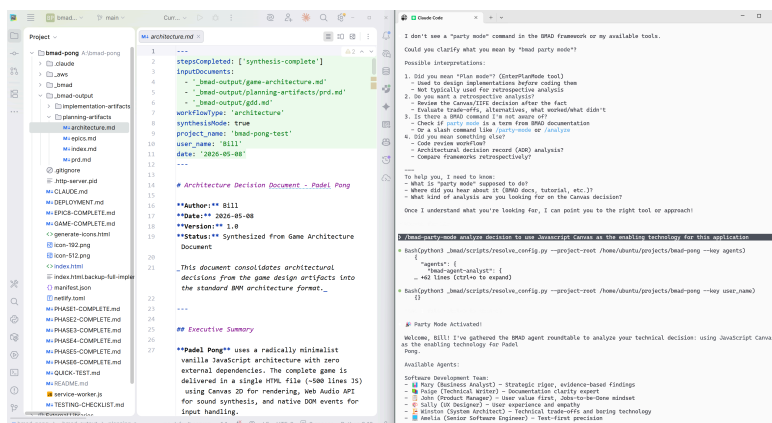


Figure 1: A development setup showing the IDE and the Claude Code CLI with BMAD on a remote Linux host

Agents, Skills, and Steps—Oh My!

We use cookies to make this site work properly. For more information, see our [Privacy Policy](#). Do you agree to us using cookies? Sure, I know how this works! - No way. Get me out of here!



personas that you can invoke to handle your prompts as part of a particular context session. BMAD is perhaps best understood as multi-agent role simulation. Spec Kit, however, relies more on the human developer to advance phases by issuing slash commands. Where BMAD agents can link workflows and autonomously orchestrate unique agents to analyze similar questions from different perspectives, Spec Kit is more tuned to allowing the human to drive each step, primarily through a simple set of commands.

An agent in this context is just another tool for conveniently shaping or engineering the context session. The artifact that expresses an agent's role and capabilities is typically a set of markdown files containing a persona/role definition, a set of skills and responsibilities, instructions and constraints (how it should behave, what it should refuse), available tasks or commands (workflows it can execute), and references to other resources (checklists, templates, other files it should use). Instruction files and prompt files sitting on a disk are inert. They only become useful when something assembles them into a context window at the right time. An agent is what does that assembly.

When we “activate” an agent, we are just injecting that agent's markdown files into the context session as additional top-of-memory foundational context. Different agents often have access to different tools, giving each one distinct properties or “personas.” Another property of the BMAD agents is that one agent can hand off to another by passing context. As we will see below, this is more significant in BMAD, which is a multi-agent methodology, as compared to

BMAD & Spec Kit Outputs

This section provides a brief overview of the artifacts employed by each of the methodologies. As shown, both methodologies facilitate what is often referred to as spec-driven development, characterized primarily by what we have discussed above in terms of context engineering and preserving provenance. While both frameworks facilitate spec-driven development, they differ in the philosophy, tools, and techniques that they each employ. BMAD, for example, relies heavily on the use of multiple agents with their own personas and specialized skills for developing the spec, whereas Spec Kit employs a single agent and is widely viewed as employing a more streamlined process with a shorter learning curve.

Table 1 shows the different artifacts that each methodology generates and relies upon for managing the context session. Items are shown from higher order functions to lower order functions. As you can see, each employs a set of markdown files that fills a set of largely similar functions. In both methodologies, higher order functions facilitated by markdown files that the agent (or agents—in the case of BMAD) load into the context session in a particular order, exploiting the `Ordering` primitive, and obtain the positional bias that artifacts loaded earlier have by virtue of being loaded in the top of the model's memory. As such, the files follow a strict cascading hierarchy where high-level documents inform and constrain low-level ones.

Table 1: Structural hierarchy of the primary file artifacts for BMAD and Spec



Global Rules	architecture.md	constitution.md	standards. Coding styles, language preferences, other opinions and other "hard landscape" elements of the project are stated at this level in both methodologies. Different: constitution.md is an immutable "law"; BMAD mixes this into architecture and role definitions.
Requirements	brief.md, prd.md	spec.md	Same: Defines what is being built. Different: BMAD separates the initial concept and scope (brief.md) from detailed functional requirements (prd.md); Spec Kit captures both in the spec.md file.
Design	architecture.md	spec.md	Same: Provides high order technical design information. Different: BMAD uses a dedicated file for system flow and data schemas; Spec Kit bundles technical design into the spec.



			epics.md is an agile construct that consists of a grouping of related features or stories; plan.md is a step-by-step implementation strategy.
Discrete Tasks	<story_name>.md	tasks.md	Same: The smallest unit of work for the AI. Different: <story_name>.md includes executable coding instructions for the Developer agent; tasks.md is often a checklist.
Organization	index.md	(folder based)	Same: Provides fine-grained context session management for low order operations. Different: BMAD uses index.md to catalog and describe every file in a directory to maintain AI context; Spec Kit relies on a folder structure to organize units of work.

Some major elements of the context session not represented by these artifacts are the activated agent as well as other prior code that might be selected into a given context session, when implementing stories (BMAD) or tasks (Spec Kit). In addition, for the AI



methodology we seek to employ as well as providing for other fundamental elements pertaining to the Claude models.

A Note on Manual Edits in the BMAD and Spec Kit Frameworks

Manual edits should primarily occur only in the top two levels of either hierarchy (Architecture /Constitution or `PRD` / `Spec`). The AI then uses these as the hard landscape for regenerating lower-level plans and tasks when requirements change. Techniques can be applied to protect manual changes from being updated by an AI agent.

A Quick Tour of the BMAD Methodology

In this section, we will take a closer look at the different types of artifacts that inform the context in the BMAD methodology and, in turn, guide the developer in generating high-quality AI-assisted code. BMAD provides a rich hierarchy of agents, workflows, and builders for your project. **Table 2** provides a summary view of this hierarchy.

Table 2: BMAD agents and other artifact types found in .claude/skills directory

Artifact Type	Examples	Description
Persona Agents	bmad-agent-*, bmad-cis-agent-*, gds-agent-*	Conversational, named, open-ended
Orchestrator	bmad-party-mode	Multi-agent coordination
Builder	bmad-create-*, bmad-update-*, bmad-delete-*	Generates, updates, or deletes artifacts



	bmad-shard-doc, etc.	meta-level
Module Builders	bmad-agent-builder, bmad-module-builder, bmad-workflow-builder	Meta: builds other skills/agents

BMAD Knows How to Party

As mentioned above, BMAD is mostly known for the many personas (i.e., agents) that it can bring to a task. BMAD can orchestrate multiple agents using a command called “Party Mode” (i.e., `/bmad-party-mode`). Party mode is an orchestration command that can spawn multiple agents to analyze a piece of work, each providing unique insights and often highlighting the tension between the different perspectives and the different choices that you can make. See **Listing 1** to see how the actual BMAD frontmatter and definition describe Party Mode and its purpose.

Listing 1: Party mode description and purpose from `.claude/skills/bmad-party-mode/SKILL.md`



```
---
name: bmad-party-mode
description: Orchestrates group discussions, enabling natural multi-agent collaboration. This is a real subagent with independent thought and requests. Party Mode, wants multiple agent discussion, roundtable, or multi-agent collaboration project.
---

# Party Mode

Facilitate roundtable discussions where agents act as **real subagents** -each spawned independently
```



Below, we will see what it looks like when I asked BMAD to throw a party around which game framework was best for the custom “Padel Pong” game that I created for this article. But first, let's look a little closer at what makes a BMAD agent an agent.

What Is a BMAD Agent?

There is a lot of talk about AI agents everywhere these days, but what is an agent really? Above, I provided one definition of what an agent is, but with BMAD, we can see clearly the tangible artifacts that make up a BMAD agent and its persona. If you look at the git repository for this project, you will see all the unique agents / personas that BMAD offers by looking for files named `bmad-agent-*` under the `.claude/skills` directory.



```
$ cd ~/projects/bmad-pong/.claude/skills
$ ls -1 | grep bmad-agent
bmad-agent-analyst
bmad-agent-architect
bmad-agent-builder
bmad-agent-dev
bmad-agent-pm
bmad-agent-tech-writer
bmad-agent-ux-designer
```

Note that the BMAD methodology installs a full suite of agents and workflows for game development. In fact, to create the “Padel Pong” demo app shown below, I employed the `/gds-create-gdd` workflow skill to leverage a workflow that factors every aspect of game development. Even with a simple game like “Padel Pong,” I spent roughly 8 hours completing the 14-step workflow—with several sub-workflows—to



above to be sure to preserve provenance in the typical BMAD way.

A quick look at an agent's frontmatter tells you a few things about agents right away. First, they are typically given formal names, and those names will be used when the output is generated by that agent. Here is what the frontmatter looks like for the BMAD Architect agent in its `SKILL.md` file:

```
---
name: bmad-agent-architect
description: System architect and technician
            asks to talk to Winston or
---
```

From there, there are usually a predictable set of sections that define the agent's unique capabilities and constraints. For the `bmad-architect-agent`, those sections include the following:

```
# Winston — System Architect
## Overview
## Conventions
## On Activation
### Step 1: Resolve the Agent Block
### Step 2: Execute Prepend Steps
### Step 3: Adopt Persona
### Step 4: Load Persistent Facts
### Step 5: Load Config
### Step 6: Greet the User
### Step 7: Execute Append Steps
### Step 8: Dispatch or Present the Menu
```

The `SKILL.md` file uses placeholder values such as `{agent identity}` and `{user name}` so that

Introducing Padel Pong

As described above, I spent several hours to leverage the BMAD agents and workflows tailored to game development. After completing the lengthy specification process with expert game development guidance, I had a brand-new version of Padel Pong that I could play immediately! **Figure 2** shows what the key planning artifacts look like in the IDE. The `implementation-artifacts` folder contains all the stories needed to implement the build. There are too many to show them all here, but I encourage you to download the project and look at them for yourself.

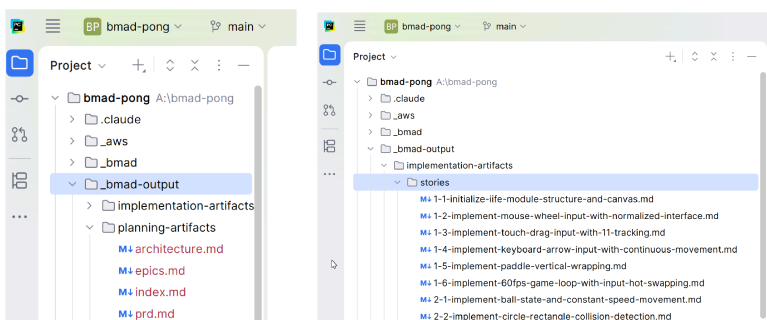


Figure 2: BMAD planning and implementation artifacts.

I then ran Party Mode (`/bmad-party-mode`) against a selected epic—namely, the epic where I had chosen the JavaScript framework and architecture. **Listing 2** shows some of the resulting output from the “always-ready-to-party” BMAD agents.

Listing 2: BMAD Party Mode analyzing choice of JavaScript Canvas



```
> /bmad-party-mode analyze decision to
enabling technology for this applicator
🎉 🎉 🎉 Party Mode Activated!
```

- 📊 Mary (Business Analyst) – Strategic findings
- 📖 Paige (Technical Writer) – Documentation
- 📋 John (Product Manager) – User validation, jobs-to-be-done mindset
- 🎨 Sally (UX Designer) – User experience
- 🏗️ Winston (System Architect) – Tech stack, boring technology
- 💻 Amelia (Senior Software Engineer) – Implementation
- 🧪 Murat (Test Architect) – Risk-based testing

Many elements that were my unique ideas for a “modern pong” have been captured. For example, since the consensus was pretty clear, I decided to go with a simple Canvas-based implementation. I also decided that the mouse wheel would be the primary controller on desktop browsers but was able to include support for touch for mobile devices as well. I also specified the sound effects, colors, and effects that happen during gameplay.

Many other decisions are reflected as well—see if you can tell how Padel Pong is different from classic Pong! **Figure 3** shows the Padel Pong start screen and some early gameplay. As exciting as that is, the most exciting part for me is that by following a disciplined approach, managing context, and preserving provenance in a structured way, I am ready to go back into any part of the game to further refine it or add new features, without losing all of the valuable information that was curated during this build process.



A Quick Tour of the Spec Kit Methodology

Where BMAD seeks to bring domain experts (i.e., agents with personas and specialized skills) with multiple perspectives and comprehensive workflows to the party, Spec Kit brings a refined process that primarily relies on the human developer to drive the process. The idea here is that you have a human product development and software engineering team for assessing the market and developing requirements, but the developer is looking to use a methodology to reduce human input and feature requests to a specification that either a human or a coding agent can implement. Again, the key benefit being sought here is managing context, preserving provenance, and creating artifacts that can be reused and refined by anyone in the future.

The Spec Kit Development Loop

The Spec Kit methodology primarily revolves around the idea that you already know what you want to do, or you are picking up a brownfield project, and you would like Spec Kit to infer what has been done in the past so that you can iterate on it. There are five phases in total, but as we will see, most of the work involves looping through the middle three phases. The five phases are:

1. **Global Rules.** (`/speckit-constitution`)

Define or update your project's guiding principles and development guidelines.

2. **Requirements.** (`/speckit-specify`) Outline



chosen tech stack.

4. **Discrete Tasks.** (`/speckit-tasks`) Generate actionable and structured task lists for development.
5. **Implement.** (`/speckit-implement`) Execute all defined tasks to build features according to your plan.

As mentioned, when working with Spec Kit, you will spend most of your time looping through requirements, execution plan, and discrete tasks. However, you can use `/speckit-constitution` at any time to revise your global rules. Likewise, you can move to the Implement step—`/speckit-implement`—at any time to generate an implementation that includes the latest feature.

When working with Spec Kit, you will see that the experience is far less chatty than working with a BMAD agent. With BMAD, you are signing up for a guided tour with a very knowledgeable tour guide. As such, you are inviting in-depth questioning and feedback from one or more experts in the areas of market research, technical frameworks, user experience, and more. With Spec Kit, you are simply asking for a methodology that helps with certain bookkeeping and other techniques around preserving provenance. There is no off-the-shelf team of chatty agents ready and eager to spend hours walking you through workflows covering every stage of software development. However, you do get well-structured outputs, a clean methodology, and the ability to bring your own thinking to either a brownfield or greenfield project very quickly.

Advertisement



Padel Pong Unleashed

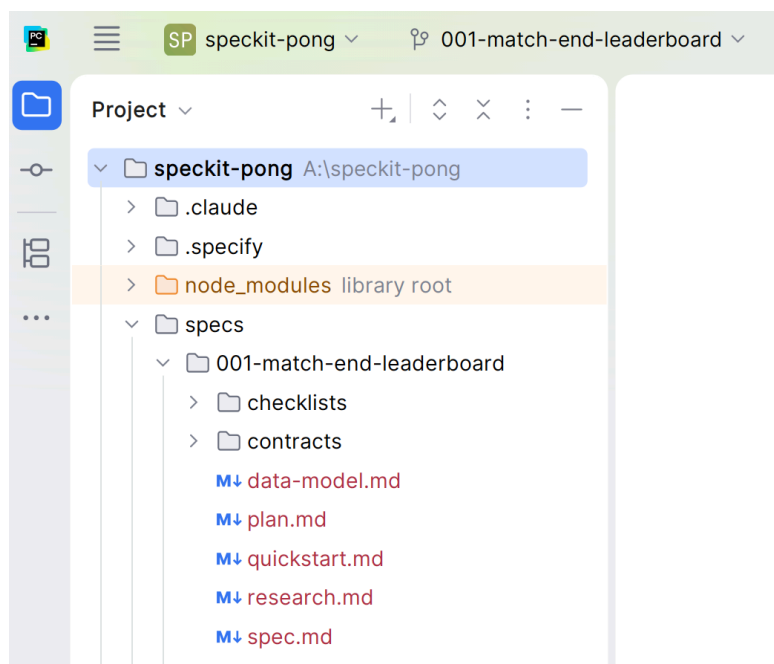
To showcase Spec Kit's strength in adding features to an existing codebase, I decided to take the resulting code from the BMAD methodology and add it—and it alone—to a new project. The goal here is to show how it is possible to take a brownfield project and get started on it using a structured AI-assisted methodology just as quickly as if you were simply vibe coding.

Of course, you are leaving a lot up to Claude Code to infer from the code itself as to why certain things exist the way that they do, and you don't immediately obtain the surgical control over the generated code that you can achieve when carefully developing the specs. However, it does demonstrate that you can get started quickly and you can begin generating artifacts that capture your decisions and preserve provenance for future sessions. Further, as the old saying goes, you can't edit a blank page. By getting started quickly in this way, you begin generating the provenance artifacts that reflect the current state and can be edited moving forward, whether to add features or just better document the current code and the product intent.

`constitution` command against just the `index.html` code file, and perhaps further refining it with some additional prompts, you are now ready to use the `/speckit-specs` command to begin a new feature. Doing so can look like this:

```
> /speckit-specify simple leaderboard th
```

As shown in **Figure 4**, that simple command will result in Spec Kit generating a new feature branch in git and proceeding to fill in all the artifacts that are needed to implement the feature! Of course, it is unlikely that the new leaderboard will show everything we want in just the way we want it, but we have just set up the full scaffolding needed to take that feature wherever we want! Moreover, your results will be preserved in a well-organized set of artifacts that can be picked up at any time to be reviewed, reused, tested, or refined by you, someone on your team, or anyone that you would like to invite to contribute to your code.



the new version (right), which implements the new `001-match-end-leaderboard` feature, overlays an initial version of our leaderboard showing the two players and even allowing some user feedback on how the gameplay was.

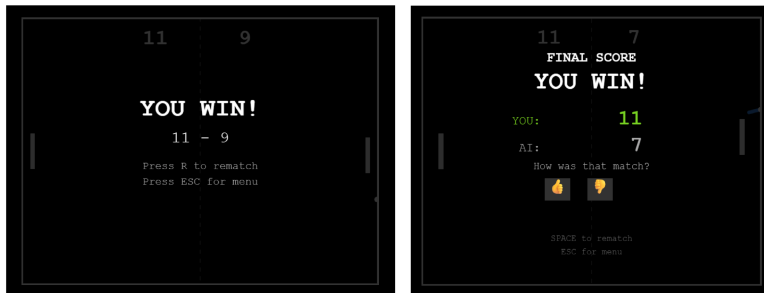


Figure 5: The original end screen (left) and the new end screen (right) showing the first version of a leaderboard.

Use AWS Bedrock for Private AI-Assisted Coding

When you are engineering context and preserving provenance, depending on the nature of the work that you are doing, there is a good chance that you are also developing high-value intellectual property while doing that work. Whether you should allow your session inputs and outputs to get shared back to the model that you are working with is an important question. Many organizations have a compelling interest in treating the work product that their product teams and developers create during AI-assisted development as proprietary and/or confidential; they want to ensure that their newly developed work is not being fed back into a public large language model.



Services such as AWS Bedrock offer a service where they make the foundation large language model available to your developers—something that would take millions of dollars in processing power alone, and a lot of time and know-how, for your organization to curate on its own—while also offering guarantees that your session inputs and outputs belong to you alone, and will not be ingested back into the large language model or otherwise used to inform responses to prompts by other users that use the model.

As stated on the AWS website, “Amazon Bedrock has a concept of a Model Deployment Account—in each AWS Region where Amazon Bedrock is available, there is one such deployment account per model provider. These accounts are owned and operated by the Amazon Bedrock service team. Model providers don't have any access to those accounts. After delivery of a model from a model provider to AWS, Amazon Bedrock will perform a deep copy of a model provider's inference and training software into those accounts for deployment. Because the model providers don't have access to those accounts, they don't have access to Amazon Bedrock logs or to customer prompts and completions.” Aside from keeping your inputs and outputs safe from being ingested into the model, AWS and Anthropic also offer certain copyright indemnifications that are also worth taking a quick look at.

Because the model providers don't have access to those accounts, they don't have access to Amazon Bedrock logs or to customer prompts and completions.



Dive Deeper Into Data Protection Policies

The most authoritative answers around the full scope and attestations of data protection can be found in [Amazon Bedrock FAQs](#) and their [data protection docs](#). For Anthropic's published statements on such topics, see **Settings > Privacy > How we protect your data and Settings > Privacy > How we use your data in the Claude console. Also see, [Is my data used for model training? \(Consumer\)](#) and [Is my data used for model training? \(Commercial\)](#) to compare use cases.**

The AWS framework is more clearly documented and has more structure that can be administered by your enterprise administrators, eliminating the chance for individual users to make mistakes when making selections for how data should be shared. For example, while Anthropic's policy asserts that session data sharing is not enabled by default, and a user would need to opt in, I was surprised when I found the "Help improve Claude" toggle enabled in my account, thereby opting me in to sharing my conversation and coding sessions back to the public model. I trust that I did indeed opt in at some point, but I simply don't recall doing so. See **Figure 6** for what the sharing session data toggle looks like for Claude Code.

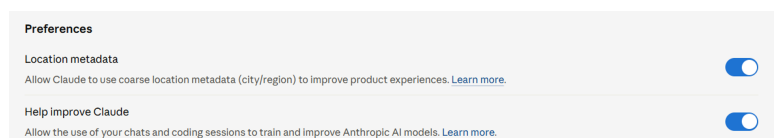


Figure 6: Anthropic setting for sharing session data back to the model is shown here as enabled. That may not be what you want.



Claude Code falls exclusively under the policies and protections for consumer customers.

An Air-gapped AI Development and Runtime Environment

For a long time now, AWS has offered support for completely private access to its resources and services. AWS began with supporting private networking pipes from external datacenters into AWS accounts in order to connect hybrid deployments and continues to this day to support the private network use case with current offerings of its `PrivateLink` endpoints in the Virtual Private Cloud (VPC).

PrivateLink allows compute and other AWS services to establish service endpoints within customer VPCs that are reachable via private IP addresses in any of the `RFC 1918` defined ranges and which are addressable over private routes—without any routing to the public internet required. Being a PrivateLink enabled service, in addition to the standard public AWS API, the AWS Bedrock service offers PrivateLink endpoints for accessing its full array of AI models sourced from multiple providers over routes that are completely isolated from the public internet. Implementing a private networking strategy like the one shown in **Figure 7** has the advantage of both reducing your attack surface and benefiting from the performance benefits of traffic remaining on the AWS networking rails.

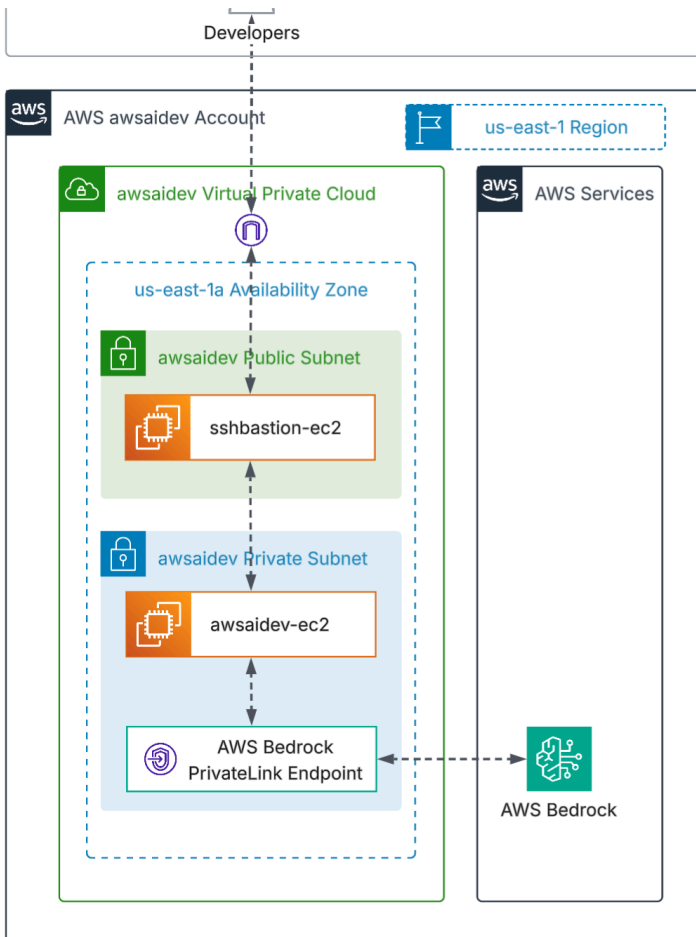


Figure 7: An air-gapped build using AWS Bedrock with PrivateLink.

Allowing Egress to the Public Internet

It is often the case that a completely air-gapped solution is not desirable, and your requirements will dictate that your development environment needs access to various resources over the public internet. In such cases, the typical enterprise solution would be to provide a route to an AWS Managed NAT Gateway, which only allows outbound connections to the internet.

However, for individuals and small teams, you could also opt to host your own NAT gateway instance on the same EC2 instance that you are using as a bastion server. Or, you could even



groups. While it might not check all of the boxes for an environment subject to strict compliance audits—and it can be more prone to configuration mistakes or certain malfeasance—such a configuration can provide a secure and fairly low-cost internet egress solution for individuals and small teams. You could even use this outbound internet capability to establish an outbound VPN link to your local router to allow easy access over a VPN tunnel from your local machine.

A Full Array of Models and Enterprise Security

As shown in **Listing 3**, AWS Bedrock supports a full array of foundation models from many of the major vendors. Some models can be significantly more costly than others and it is often important for enterprises to be able to put controls around which models their users are able to access. Because we are using an EC2 instance in AWS, we can take advantage of the tight controls enabled by AWS's role-based permissions system. Our EC2 IAM role profile, for example, contains permissions for just a single model.

Listing 3: Foundation models available from AWS Bedrock.



```
$ aws bedrock list-foundation-models --r  
| grep modelId | sort  
  
"modelId": "ai21.jamba-1-5-large-v1:0"  
"modelId": "ai21.jamba-1-5-mini-v1:0",  
"modelId": "amazon.nova-2-lite-v1:0",  
"modelId": "amazon.nova-2-lite-v1:0:25
```



```
"modelId": "amazon.nova-lite-v1:0:300k",  
"modelId": "amazon.nova-micro-v1:0",  
"modelId": "amazon.nova-micro-v1:0:128",  
"modelId": "amazon.nova-micro-v1:0:24k",  
"modelId": "amazon.nova-premier-v1:0",
```

If we wanted, we could allow newer, more expensive Claude models, or models from any of the vendors shown in **Listing 3**. **Listing 4** shows the permissions needed to allow just one model. Any of the model IDs could be added by an authorized AWS user to allow more options. When this role is assigned to a group, anyone in the groups would obtain the same permissions.

Listing 4: AWS IAM role permissions for accessing AWS Bedrock.



```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "AllowBedrockStreamir",  
      "Effect": "Allow",  
      "Action": [  
        "bedrock:InvokeModel",  
        "bedrock:InvokeModelWith",  
      ],  
      "Resource": [  
        "arn:aws:bedrock:*::four",  
        "arn:aws:bedrock:*:*:inf",  
      ]  
    },  
    {  
      "Sid": "AllowBedrockDiscover",  
      "Effect": "Allow",
```

In short, AWS Bedrock provides a very robust and secure architectural framework for your data—both



for compliance requirements or other purposes can be achieved. As shown, model access can be controlled by centralized permission policies, and accidental data sharing is not susceptible to human error when setting up a local machine.

Closing Remarks

I hope that this article helped to pull back the curtain on some of the mystery surrounding the tools and techniques that you can apply when generating AI-assisted code. Like everything else, AI-assisted coding can be done well, or poorly. By leveraging a good framework, your knowledge can be amplified and your productivity will multiply. By focusing on being very intentional in managing the context sessions you generate each step of the way, the resulting output will be more predictable and better reflect your expectations. Also, by using a structured approach to engineering context and preserving provenance during all stages, the entirety of every choice and outcome can be reviewed, reused, tested, and refined by others later, thereby empowering your team and even others beyond your organization. And ***that*** is professional-grade.

This article was filed under:

AI

This article was published in:





All good things must come to an end...

Luckily, this isn't one of them!

Click to get a **free trial subscription** to *CODE Magazine* and read more great articles like this one!

Have additional technical questions?

Get help from the experts at *CODE Magazine* - sign up for our free hour of consulting!

Contact CODE Consulting at techhelp@codemag.com.

Contact

6605 Cypresswood Dr.
Suite 425
Houston TX 77379
USA

📞 Voice: +1 (832) 717-4445

✉ Email:
info@codemag.com

© 1993 - 2026 EPS
Software Corp.

[Privacy Policy](#)

Site Map

Consulting

Consulting Home
Artificial Intelligence (AI)
Custom Application Development
Executive Briefing (AI)
Low-Code/No-Code
Cyber Security
Legacy Maintenance and Conversion
Project Rescue
CODE Labs

Training

Training Home
CODE Presents
State of AI
State of Fabric
State of .NET
Global AI
Classes
Mentoring

Staffing

We use cookies to make this site work properly. For more information, see our [Privacy Policy](#). Do you agree to us using cookies? Sure, I know how this works! - No way. Get me out of here!



[All Issues](#)
[Subscribe](#)
[Mobile](#)
[My \(Digital\) Magazines](#)
[Where is my Magazine?](#)
[My Subscriber Account](#)
[Advertise](#)
[Write](#)

Framework

[Framework Home](#)
[Get Started & Documentation](#)
[Download](#)
[Support & Services](#)

VFP Conversion

[VFP Conversion Home](#)
[Services](#)
[Tools](#)
[Articles](#)
[Fox End of Life](#)
[Contact Us](#)